

Actividad 7. Ética y Ciclo de Vida del ML con Meta AI.

Prompt 1. ¿Qué es el sesgo algorítmico en Machine Learning y cómo pueden los datos de entrenamiento introducir discriminación sistémica en un modelo? Proporciona un ejemplo real documentado donde un modelo de ML haya causado un daño social concreto (en contratación, crédito, justicia penal, etc.) y explica qué tipo de sesgo lo originó y cómo podría haberse mitigado.

El sesgo algorítmico es cuando un modelo de machine learning produce resultados injustos porque aprende patrones que reflejan prejuicios sociales previos, no una verdad neutral.

IBM lo define así: ocurre cuando los algoritmos "producen resultados injustos o discriminatorios, reflejando o reforzando prejuicios socioeconómicos, raciales y de género". No es un bug matemático, es el modelo copiando el mundo tal como está en los datos.

Cómo los datos de entrenamiento meten discriminación

Los datos no son un espejo limpio. Introducen sesgo en tres momentos clave:

- **Recolección histórica.** Si entrenas con decisiones pasadas, el modelo aprende esas decisiones. La discriminación algorítmica "se presenta cuando los sistemas perpetúan inequidades debido a sesgos originados en la etapa de recolección, limpieza o depuración de datos".
- **Representación.** Cuando un grupo está subrepresentado, el modelo no tiene ejemplos suficientes para aprender bien de él, y optimiza para la mayoría.

- **Etiquetado y proxies.** Aunque quites la variable "género", el modelo encuentra sustitutos: palabras, universidades, hobbies que correlacionan con género.

El resultado: el sistema no inventa el prejuicio, lo automatiza y lo escala.

Ejemplo real documentado: el reclutador de Amazon (2014-2018)

Es el caso más citado en contratación porque fue interno, a gran escala, y terminó cerrado.

- Amazon construyó desde 2014 un motor para puntuar currículums de 1 a 5 estrellas, con la idea de "darle 100 currículums y que escupa los 5 mejores".
- Para entrenarlo usó "patrones en resumes enviados a la compañía durante un periodo de 10 años. La mayoría provenía de hombres, reflejo del dominio masculino en la industria tech".
- Para 2015 el equipo vio que no era neutral: "el sistema de Amazon se enseñó a sí mismo que los candidatos masculinos eran preferibles. Penaliza los currículums que incluían la palabra 'women's', como en 'women's chess club captain'. Y degradó a graduadas de dos universidades solo para mujeres".
- No era solo una palabra. El modelo favorecía verbos más comunes en CV masculinos, como "executed" y "captured".
- Amazon intentó editar esas palabras, pero "no había garantía de que las máquinas no idearían otras formas de clasificar candidatos que pudieran resultar discriminatorias".
- Resultado: el equipo fue disuelto a inicios de 2018. Los reclutadores habían mirado las recomendaciones, pero "nunca confiaron únicamente en esos rankings".

Daño social concreto: aunque no se contrató automáticamente, miles de mujeres en tecnología vieron sus CV sistemáticamente degradados por un sistema usado en la empresa más grande de e-commerce. Es discriminación por proxy a escala industrial, que refuerza la barrera de entrada histórica.

Tipo de sesgo que lo originó:

1. **Sesgo histórico.** El pasado de contrataciones masculinas se convirtió en "verdad" para el modelo.
2. **Sesgo de representación.** Dataset con ~70-80% hombres en roles técnicos, mujeres como ruido estadístico.
3. **Sesgo de medición por proxy.** Género eliminado explícitamente, pero reintroducido vía lenguaje y universidades.

No fue un caso aislado. En justicia penal, ProPublica encontró el mismo patrón con COMPAS: "defendientes negros que no reincidieron en dos años tenían casi el doble de probabilidad de ser clasificados como alto riesgo (45% vs 23%)", mientras blancos reincidentes eran etiquetados como bajo riesgo casi el doble de veces.

Cómo podría haberse mitigado (y cómo se hace ahora)

Amazon intentó el parche fácil, borrar palabras, pero eso no corrige la distribución subyacente. La mitigación real empieza antes del modelo:

- **Auditoría de datos antes de entrenar.** Medir la proporción por género, raza, universidad. Si tu dataset de 10 años es 85% hombres, no entrenes hasta reequilibrar o estratificar.
- **Datos diversificados y sintéticos.** IBM señala que la mitigación "implica gobernanza ética, datos diversificados y transparencia". En práctica: sobremuestrear CV femeninos exitosos, usar generación de datos para cubrir huecos, no solo "quitar la palabra women".
- **Eliminar proxies y probar adversarialmente.** Entrenar un clasificador secundario que intente predecir género a partir de las features. Si lo logra, tu modelo principal todavía tiene fuga de información.
- **Métricas de equidad durante el entrenamiento, no después.** Igualdad de oportunidad (tasa de falsos negativos similar entre grupos), paridad demográfica. Amazon solo miró precisión global.
- **Human-in-the-loop con explicabilidad.** No puntuar de 1 a 5 sin explicación. Mostrar qué n-gramas pesaron, permitir apelación.

- **Gobernanza.** Comité externo, registro de versiones de datos, y la regla de LinkedIn citada en el reportaje: "Ciertamente no confiaría en ningún sistema de IA hoy para tomar una decisión de contratación por sí solo".

La lección es simple: los modelos no corrigen la historia, la comprimen. Si alimentas 10 años de sesgo, obtienes un reclutador que escribe ese sesgo en código y lo ejecuta miles de veces por segundo. Mitigar no es borrar una palabra, es rediseñar el pipeline de datos con la equidad como restricción, no como afterthought.

Conclusiones: Es importante proyectar los posibles sesgos o problemas que pueda tener el modelo al entrenarlo con un cierto tipo de datos o mas bien pasando por alto las categorías específicas de estos. El prepararnos para los posibles resultados forma parte de la prevención de resultados generados por los algoritmos.

Prompt 2. ¿Qué es el data leakage (fuga de datos) en Machine Learning y por qué es uno de los errores más comunes y peligrosos al construir modelos? Explica la diferencia entre fuga de datos en el preprocesamiento (como escalar con datos de prueba) y fuga de datos conceptual (como incluir variables que en producción no estarían disponibles), y cómo prevenirlos.

La fuga de datos es cuando tu modelo hace trampa sin que te des cuenta, porque le dejas ver parte de las respuestas del examen mientras estudia. En entrenamiento obtiene métricas espectaculares, pero en producción se desploma, porque esa información extra ya no existe.

Es tan común y peligroso por tres razones: primero, es invisible en el código, no lanza error. Segundo, infla el score y te da falsa confianza para desplegar. Tercero, suele venir de buenas intenciones, como limpiar bien los datos o usar muchas variables.

Los dos sabores de fuga

No es un solo error. Hay que distinguirlos porque se previenen de forma distinta.

Tipo	Qué se filtra	Ejemplo clásico	Cuándo aparece	Señal de alerta
Preprocesamiento	Estadísticas del test o validación	Hacer <code>StandardScaler.fit</code> en todo el dataset antes de <code>train_test_split</code>	Notebooks donde primero limpias todo y luego divides	Validación 0.98 AUC, test real 0.72
Conceptual o target leakage	Información del futuro que no tendrás en el momento de predecir	Incluir "fecha de último pago" para predecir si un cliente va a pagar	Diseño de features sin pensar en el tiempo $t=0$	Una variable tiene importancia absurda y el modelo parece perfecto

1. Fuga en el preprocesamiento

Aquí el problema es el orden. Si calculas la media, la desviación, imputas nulos, haces target encoding o SMOTE usando todo el dataset, le estás contando al modelo cómo es el conjunto de prueba.

Código que filtra:

```
scaler = StandardScaler().fit(X) # aprende de train + test
X_scaled = scaler.transform(X)
X_train, X_test = train_test_split(X_scaled)
```

Código correcto:

```
X_train, X_test = train_test_split(X)
scaler = StandardScaler().fit(X_train) # solo aprende de train
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Lo mismo pasa con imputación de medianas, normalización de texto, o hacer oversampling antes del split. En cross-validation, el error clásico es ajustar el preprocesador fuera del loop.

2. Fuga conceptual

Aquí el código está bien, pero el feature es imposible en producción. Es un error de negocio, no de sklearn.

Ejemplos reales:

- Predecir fraude usando "tiempo que tardó la investigación", dato que solo existe después del fraude
- Predecir abandono de clientes con "días desde la cancelación"
- En salud, usar un resultado de laboratorio tomado después del diagnóstico
- En series temporales, incluir ventas del mismo día para predecir la mañana

El modelo no está aprendiendo patrones, está leyendo el futuro disfrazado de variable.

Cómo prevenirlos sin paranoia

Piensa en una línea de tiempo. En $t=0$ haces la predicción. Solo puedes usar lo que existiría en $t=0$.

1. Divide primero, transforma después. Haz `train_test_split` o un split temporal antes de tocar cualquier estadística. Nunca toques `X_test` hasta el final.

2. Encierra todo en Pipelines. En scikit-learn, el Pipeline garantiza que el fit ocurre solo en train dentro de cada fold.

```
pipe = Pipeline([
    ('imputer', SimpleImputer()),
    ('scaler', StandardScaler()),
    ('model', LogisticRegression())
])
cross_val_score(pipe, X_train, y_train)
```

3. Para datos temporales, usa validación temporal. No hagas shuffle. Entrena con pasado, valida con futuro. `TimeSeriesSplit` evita que el modelo vea el lunes para predecir el domingo.

4. Haz auditoría de features. Para cada columna pregúntate: "¿esta variable estaría disponible en el momento exacto de la predicción en producción?" Si la respuesta es no o depende del target, elimínala.

5. Separa transformaciones que usan y. Target encoding, WoE, o cualquier codificación basada en la etiqueta debe calcularse solo con el fold de entrenamiento, nunca con todo el dataset. Usa encoders con cross-validation interna.

6. Desconfía de scores demasiado buenos. Si pasas de 0.82 a 0.99 solo por añadir una variable, casi seguro es fuga conceptual. Revisa la importancia de features, suele delatar al culpable.

La fuga no significa que seas descuidado, significa que estás optimizando en un laboratorio que no refleja la calle. Blindar el pipeline y pensar en $t=0$ es lo que separa un modelo de notebook de uno que sobrevive en producción. ¿Quieres que revisemos un pipeline tuyo para buscar puntos de fuga típicos?

Conclusiones: Hay que ser precavidos y seguir un flujo de trabajo ordenado para tener precaución con los cálculos realizados antes de la separación del data set. Una vez obtenidos los resultados hay que analizarlos y si algo sale extremadamente bien dudar de ello (la vida real no es así).

Prompt 3. ¿Qué es MLOps y por qué es fundamental para llevar un modelo de Machine Learning del entorno de investigación a producción? Explica los conceptos de data drift, model drift y model monitoring, y describe qué estrategias se utilizan en la industria para detectar y responder cuando un modelo comienza a degradarse en producción.

MLOps es lo que convierte un experimento de notebook en un servicio que no se rompe cada vez que el mundo cambia. No es una herramienta, es una metodología DevOps aplicada a machine learning para desplegar y mantener modelos en

producción, integrando IA, datos y operaciones para asegurar confiabilidad y cumplimiento.

En la práctica, MLOps es un enfoque que integra DevOps y ML para automatizar la creación, implementación y monitoreo de modelos, optimizando tiempo, recursos y precisión. Conecta desarrollo y operaciones, permite automatizar flujos, garantizar reproducibilidad y cumplir normativas.

¿Por qué es fundamental para pasar de investigación a producción?

En investigación entrenas con datos históricos estáticos. En producción los datos llegan vivos, sucios y cambiantes.

MLOps resuelve ese salto porque:

- **Orquesta el ciclo completo**, desde acondicionamiento de datos, entrenamiento, validación y despliegue, con foco en eficiencia, reproducibilidad y escalabilidad.
- **Introduce CI/CD para modelos**, no solo para código: control de versiones de datos, de features y de artefactos, pruebas automáticas antes de desplegar.
- **Garantiza colaboración** entre científicos de datos, ingenieros y TI, que es donde normalmente se rompen los proyectos.
- **Atiende requisitos reales**: validación del modelo, cumplimiento regulatorio y eficiencia operativa, que un notebook no cubre.

Sin MLOps, el modelo perfecto del viernes se degrada el lunes y nadie lo nota hasta que pierde dinero.

Data drift, model drift y concept drift, sin humo

Model drift (deriva del modelo) es la degradación del rendimiento por cambios en los datos o en las relaciones entre variables de entrada y salida. Es el paraguas.

Dentro hay dos culpables principales:

- **Data drift (deriva de datos):** cambia la distribución estadística de los datos de entrada respecto al entrenamiento. También se describe como desplazamiento de covariables, cuando la distribución subyacente de los inputs cambió. Ejemplo: tu modelo de fraude entrenado en 2023 ve en 2025 patrones de ataque nuevos, la feature "monto nocturno" ahora tiene otra forma.
- **Concept drift (deriva conceptual):** cambia la relación entre las entradas y la variable objetivo, el algoritmo empieza a dar respuestas incorrectas porque las definiciones ya no son válidas. El Henry Blog lo resume bien: en data drift cambia el input, en concept drift cambia la función de la realidad, el modelo recibe datos familiares pero la respuesta correcta ha cambiado. Ejemplo: una hipoteca que antes era bajo riesgo ahora es alto riesgo por nueva regulación.

IBM añade un tercer tipo práctico: **cambio de datos ascendentes**, cuando el pipeline cambia unidades, monedas o esquemas, y el modelo no fue creado para eso.

Model monitoring: qué se vigila realmente

Monitoreo no es mirar accuracy una vez al mes. Es observabilidad continua de dos capas:

1. Métricas de calidad del modelo

- Accuracy, F1, precision/recall en clasificación
- RMSE, MAE en regresión Si caen bajo umbral de negocio, ya estás tarde, pero es la señal final.

2. Métricas de distribución de datos

Aquí detectas el problema antes de que pegue en el negocio. El blog de Henry lo explica como la detección temprana: comparar producción contra la línea base de entrenamiento.

Estrategias que usa la industria para detectar y responder

No hay magia, hay estadística, automatización y disciplina de despliegue.

Detección estadística continua

- **KS-Test:** mide si dos muestras vienen de la misma distribución, no requiere supuestos paramétricos. En MLOps se usa para comparar entrenamiento vs producción; un KS alto indica drift significativo.
- **Jensen-Shannon Divergence (JSD):** mide similitud entre distribuciones de probabilidad, si la distancia crece, dispara alerta.
- **Wasserstein (Earth Mover's Distance):** compara entrenamiento con nuevos datos, destaca en relaciones complejas y maneja outliers.
- **PSI (Índice de Estabilidad Poblacional):** compara distribución de features categóricas en dos conjuntos, un PSI alto indica deriva.

La mejor práctica es automatizar esta detección, porque la precisión puede degradarse en días después del despliegue. Las organizaciones usan detectores que alertan automáticamente cuando la precisión cae bajo umbral, y hacen monitoreo continuo comparando datos de producción y entrenamiento en tiempo real.

Pipeline de respuesta, no solo alertas

1. **Captura en vivo:** herramientas como MLflow, Kubeflow, SageMaker o Vertex AI interceptan y registran entradas y salidas en tiempo real.
2. **Comparación programada:** cada hora o día se ejecutan KS-Test, JSD contra la línea base.
3. **Alerta con diagnóstico:** no solo "hay drift", sino "la feature ingreso mensual se desvió 15%", para reducir investigación.
4. **Retraining automático:** el pipeline toma datos recientes validados, reentrena, valida en staging y solo si pasa, promueve.
5. **Despliegue seguro:** shadow deployment o canary deployment para reemplazar el modelo obsoleto minimizando riesgo.

Gobernanza y causa raíz

- Análisis temporal para ver si la deriva fue repentina, estacional o gradual.
- Entorno unificado para gestionar modelos, métricas y linaje de datos desde un panel central, lo que ayuda a detectar y corregir deriva y sesgo en producción.

- Verificación de inputs: a veces no es drift, es que entrenaste con escáneres de alta resolución y en producción solo hay baja resolución.

En resumen, MLOps no evita que el mundo cambie, evita que tu modelo se quede hablando un idioma antiguo. Detectas con estadística, respondes con automatización, y mantienes el valor porque el pipeline está diseñado para envejecer con gracia, no para romperse en silencio.

Conclusiones: MLOps evita que el modelo se quede atascado con conocimiento antiguo. A partir de estadísticas, automatización y mantenimiento se proyecta el valor sobre el pipeline que se diseñe. Data drift y concept drift pasan siempre porque el mundo cambia, no porque el modelo sea malo. La clave es monitorear continuamente las distribuciones, detectar con pruebas como KS o PSI antes de que caiga la precisión, y tener un pipeline que reentrene y despliegue solo. Sin eso, el modelo envejece en silencio; con eso, se adapta y sigue siendo útil.